



Programação Avançada 2025-26

[13] Padrões de Software | Observer

Bruno Silva, Patrícia Macedo

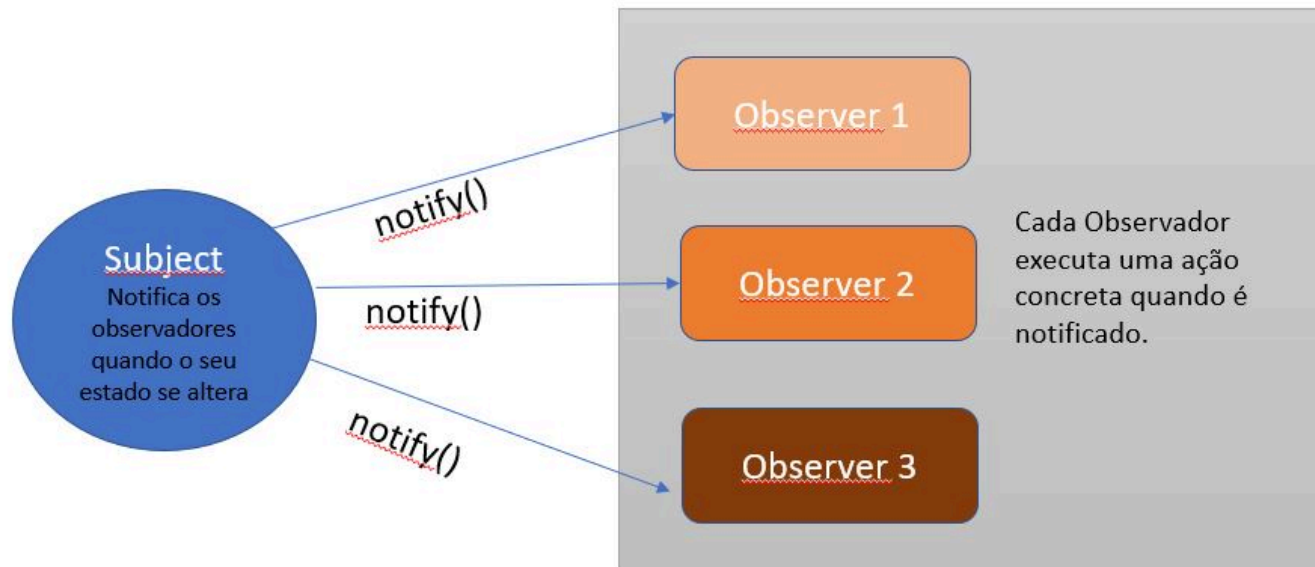
Sumário



- O Padrão Observer
- Exemplo de Aplicação

Observer | Problema

- Pretende-se assegurar que, quando um objecto **muda de estado**, um número de objectos **dependentes** é **notificado automaticamente**.

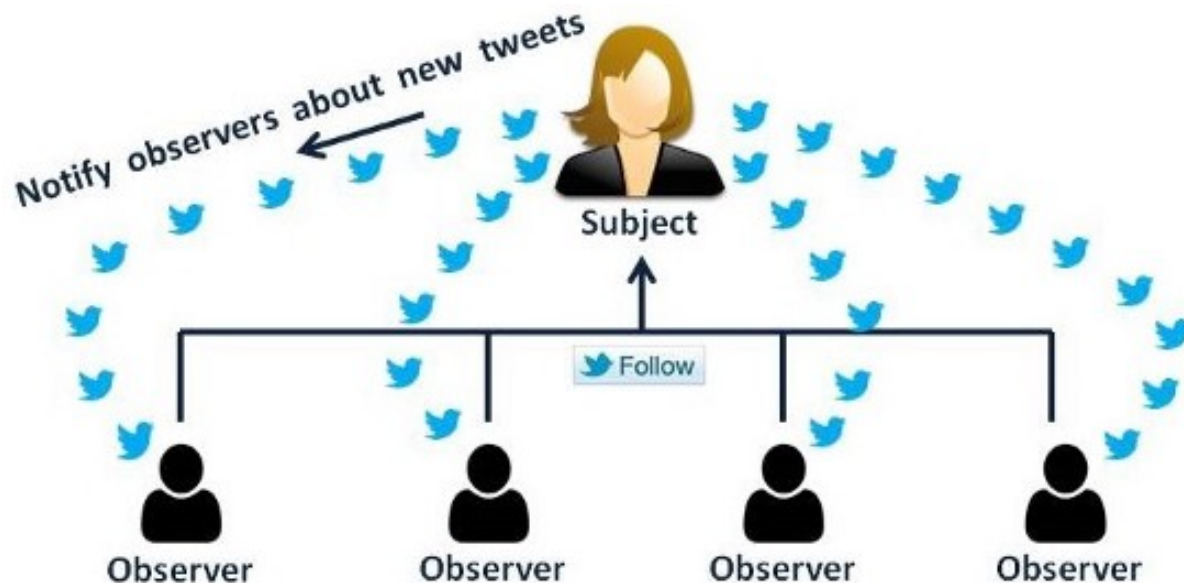


- O que cada objeto faz quando é notificado, depende do **problema**.

Observer | Enquadramento 🤔

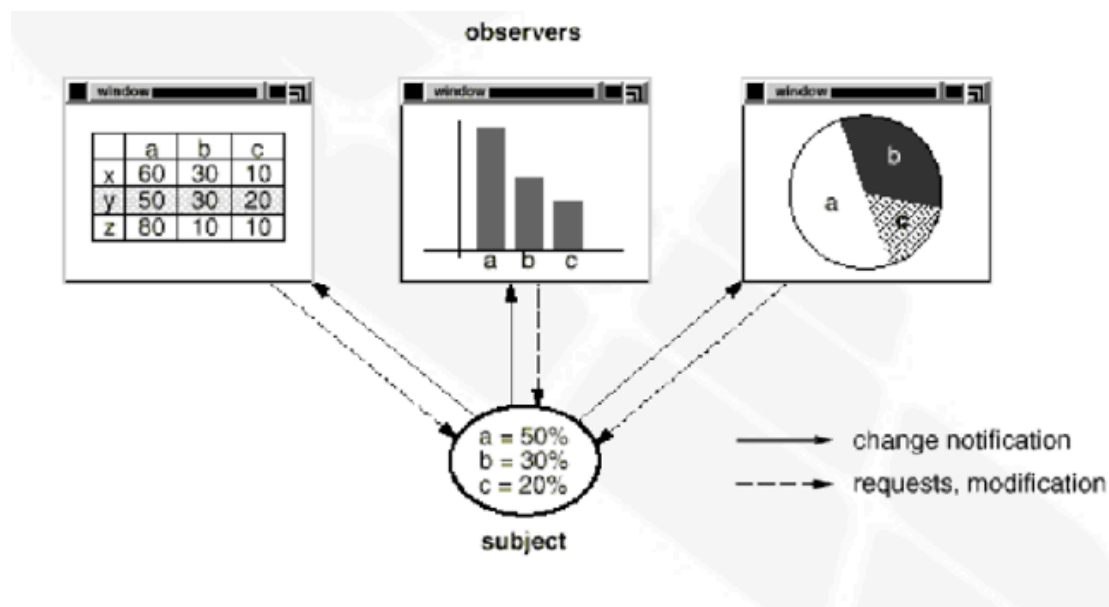
===

E.g., notificar um conjunto de susbcritores de uma plataforma que uma nova versão foi lançada.

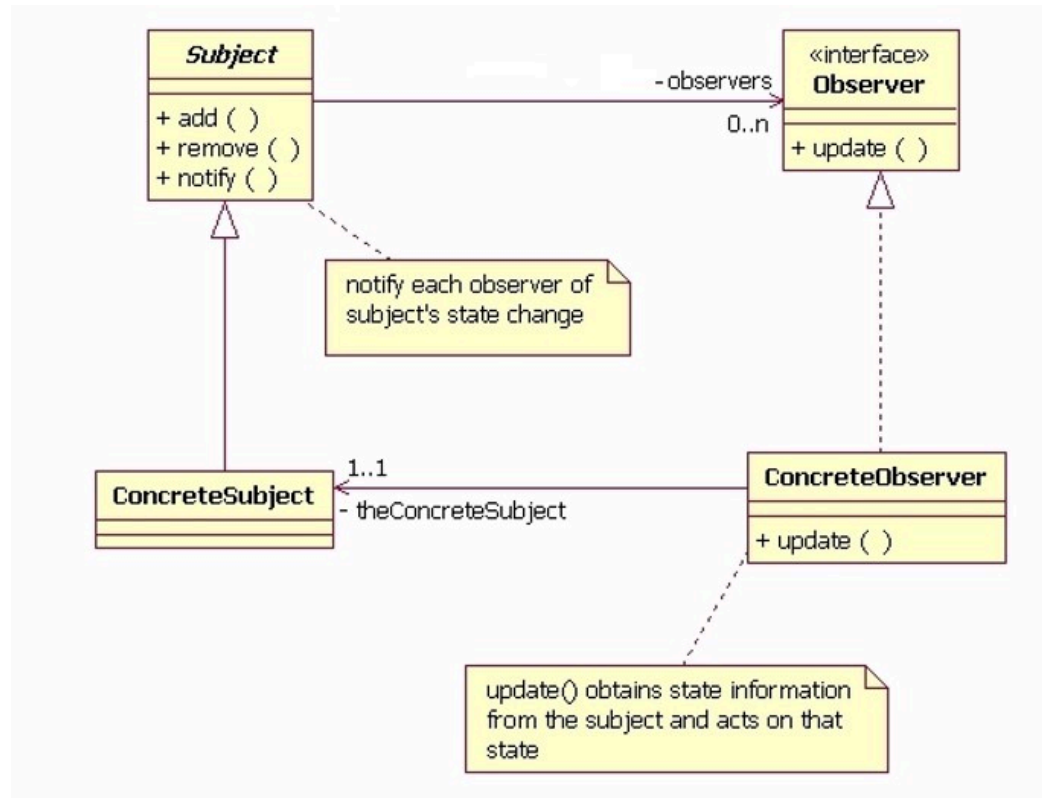


Observer | Enquadramento 🤔

E.g., atualizar a visualização dos dados automaticamente cada vez que os dados são alterados.



Observer | Solução



Observer | Participantes

- **Subject**
 - Classe *abstrata* que contém funcionalidade para adicionar/remover instâncias de `Observer` ; Portanto:
 - Um *subject* conhece os seus *observadores*;
 - Sabe notificar os *observadores* através do método `notify()`
- **ConcreteSubject**
 - "Estende" de `Subject` ;
 - Adiciona estado/comportamento adicional ao seu propósito;
 - Pode enviar notificação aos observadores, quando o seu estado se altera.

Observer | Participantes

- **Observer**
 - Define uma *interface* que "obriga" à implementação do método `update()` ; este será invocado pelo **ConcreteSubject** e representa a "notificação".
- **ConcreteObserver**
 - Implementa a interface **Observer**, para que receba as "notificações";
 - (opcional) Mantém uma referência para o objecto **ConcreteSubject**.

Observer | Nota de implementação em JAVA

- Um dos problemas que levanta a utilização do padrão Observer, é o facto de ele definir uma classe abstrata Subject em vez de uma interface.
- Como **não existe em Java herança múltipla** isso dificulta quando queremos tornar uma subclasse (de outra hierarquia) "Observable".
- Por esta razão vamos implementar uma variante do padrão Observer onde é acrescentado uma interface Observable.

Observer | interface Observable

```
public interface Observable {  
    /**  
     * Attach observers to the subject.  
     * @param observers to be attached  
     */  
    public void addObserver(Observer... observers);  
    /**  
     * Attach observers to the subject.  
     * @param observer to be removed  
     */  
    public void removeObservers(Observer observer);  
  
    /**  
     * notify all observer  
     * @param object, argument of update method  
     */  
    public void notifyObservers(Object object);  
}
```

Observer | Classe Abstrata Observable

```
public abstract class Subject implements Observable{
    private List<Observer> observerList;

    public Subject() {
        this.observerList = new ArrayList<>();
    }

    @Override
    public void addObservers(Observer... observers) {
        for (Observer obs : observers) {
            if (!observerList.contains(obs))
                this.observerList.add(obs);
        }
    }

    @Override
    public void removeObservers(Observer observer) {
        this.observerList.remove(observer);
    }

    @Override
    public void notifyObservers(Object obj) {
        for (Observer observer : this.observerList)
            observer.update(obj);
    }
}
```

Observer | Observer

```
public interface Observer {  
    /**  
     * When a observer is notified execute this method  
     * @param obj - argument of the method  
     */  
    void update(Object obj);  
}
```

Observer | Exemplo de Aplicação

Vamos aplicar o Padrão Observer, para fazermos a separação entre a lógica da aplicação e a interface com o utilizador.

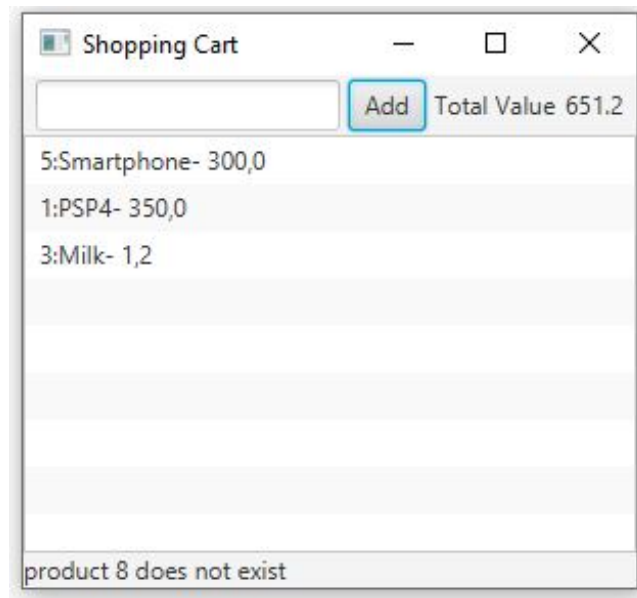
- Consideremos que o pacote Model contem as classes que contem as operações que manipula os dados.
- Consideramos que o pacote View tem as classes responsáveis por mostrar uma vista sobre os objecto do model e receber e processsar a interação do utilizador.
- Sempre que as classes do Model modificam o seu seu estado, notificam a View para atualizar a vizualização da informação.

NOTA - Poderá consultar o exemplo completo aqui:

https://github.com/estsetubal-pa-geral/ModelViewShoppingCart_Template

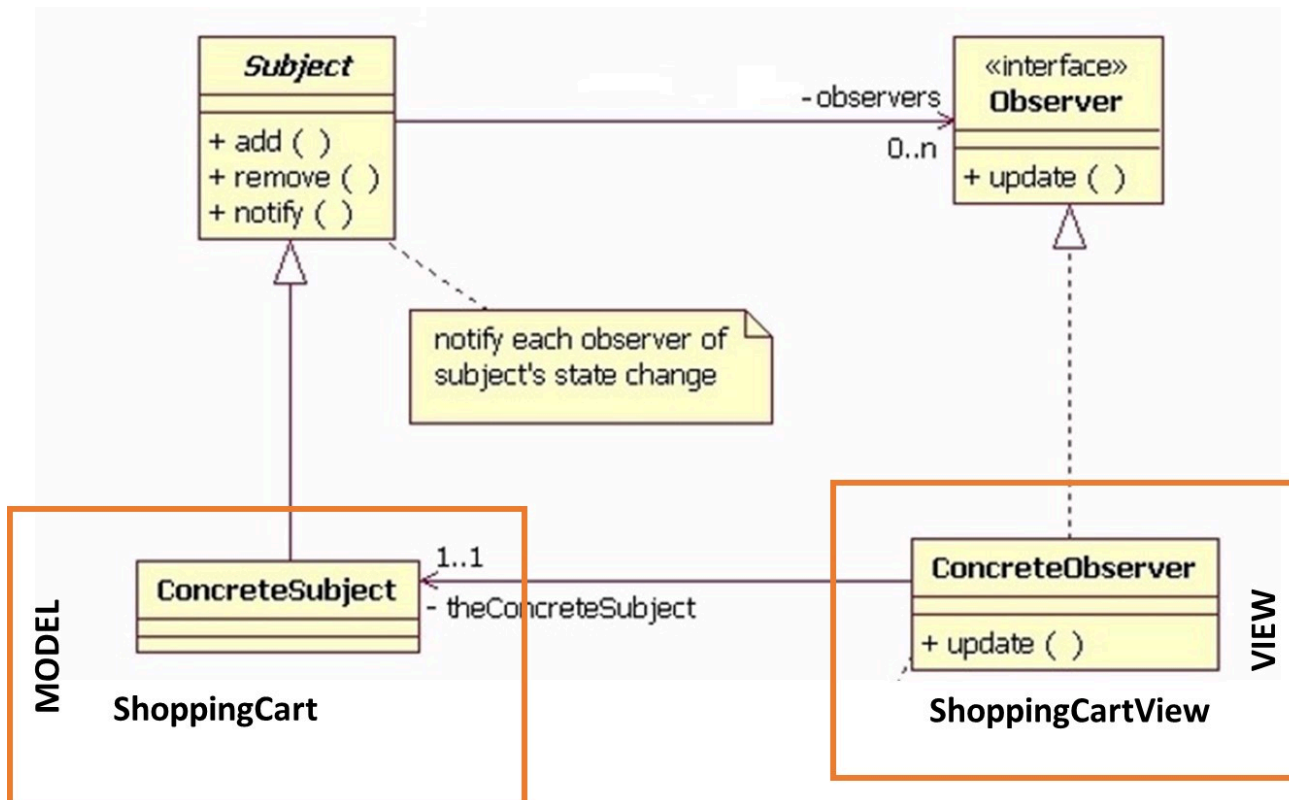
Observer | Exemplo de Aplicação

- Pretende-se realizar uma aplicação para gerir a compra de produtos online - " carrinho de compras" que tem a seguinte interface.
- Cada vez que se carrega no botão Add, um novo produto é adicionado à lista e o valor total atualizado



Observer | Exemplo de Aplicação

SOLUÇÃO



Observer | Exemplo de Aplicação

- A classe encarregue de gerir a interface `ShoppingCartView` implementa a interface `Observer`.
- E no método `update` atualiza as componentes do painel, `listProductView` e `lblCount` (apresenta o valor total do carrinho).

```
public class ShoppingCartUI extends VBox implements Observer {  
    ....  
    @Override  
    public void update(Object o) {  
        listProducts = model.getProducts();  
        this.listProductsView.getItems().clear();  
        listProductsView.getItems().addAll(listProducts);  
        lblCount.setText("" + model.getTotal());  
    }  
}
```


Observer | Exemplo de Aplicação

- A classe encarregue de implementar a lógica do carrinho de compras `ShoppingCart` estende de 'Subject'
- E cada vez que um novo elemento é adicionado ao carrinho de compras, os observadores são notificados.

```
public class ShoppingCart extends Subject {  
    // other code ....  
    public void addProduct(int id) throws ShoppingCartException {  
        Product p= ProductListFactory.createProduct(id);  
        addProduct(p);  
        notifyObservers(null);  
    }  
}
```

Observer | Exemplo de Aplicação

- É necessário agregar os vários componentes do padrão. Ou seja adicionar o uma instancia do `ShoppingCartView` como observador do `ShoppingCart`.

```
public void start(Stage primaryStage) {  
  
    ShoppingCart model = new ShoppingCart("Bruno");  
    ShoppingCartUI view = new ShoppingCartUI(model);  
    model.addObserver(view);  
  
    BorderPane window = new BorderPane();  
    window.setCenter(view);  
    Scene scene = new Scene(window, 300, 250);  
    primaryStage.setTitle("Shopping Cart");  
    primaryStage.setScene(scene);  
    primaryStage.show();  
}
```

Observer | Referências

Observer

- <https://refactoring.guru/design-patterns/observer>
- <https://www.youtube.com/watch?v=-oLDJ2dbadA>
- <https://www.youtube.com/watch?v=wqD4fOiGep4>